

OpenLDAP Performance Test Report

Symas OpenLDAP 2.6.13 on RHEL 9 -- Master/Replica -- Confidential

1. Executive Summary

A comprehensive performance test was conducted on a Symas OpenLDAP 2.6.13 master/replica cluster deployed on AWS (m5a.4xlarge: 16 vCPU, 64 GB RAM) running RHEL 9.8. Tests included login throughput (7 tuning iterations), 500,000-user write load, stress ramp, and replication sync measurement.

- Key findings:
- Optimal config: 32 threads, 512 MB LMDB cache, writemap/nometasync flags
 - Read throughput improved 23% (74.4 -> 91.6 ops/sec), p50 latency reduced 78%
 - Write speed: 390 adds/sec via streaming ldapadd (500K in 21 minutes)
 - Replica sync: 275 entries/sec sustained during write load
 - 600K users occupy 637 MB LMDB (~1 KB/user)

2. Hardware Configuration

Instance Type	m5a.4xlarge (AMD EPYC)
vCPU	16 cores
RAM	64 GB
Root Disk	50 GB gp3 (12,000 IOPS, 500 MB/s)
Data Directory	/var/symas/openldap-data/example/
OS	RHEL 9.8 (Plow)
OpenLDAP	Symas OpenLDAP 2.6.13-3
Region	AWS us-west-2
VPC	10.40.0.0/16 (perf-test)
Nodes	1 master + 1 replica + 2 load generators
Instance cost	~\$0.75/hr per LDAP node
Master IP	34.208.189.170 (private 10.40.1.10)
Replica IP	35.88.141.50 (private 10.40.2.10)
Load-gen 1	44.247.77.105 (JMeter + ldapsearch)
Load-gen 2	35.94.20.201
Topology	syncrepl refreshAndPersist, StartTLS
Base DN	dc=eab,dc=bank,dc=local
Admin DN	cn=admin,dc=eab,dc=bank,dc=local
Replicator DN	cn=replicator,dc=eab,dc=bank,dc=local

3. Read Performance Tuning (Login Load)

Seven iterations were tested with a concurrent LDAP load tester simulating login flow: service bind -> search user by uid -> user bind. Each iteration ran for 60-120 seconds at 100 target ops/sec.

Iter	Config	Ops/sec	p50 (ms)	p95 (ms)	p99 (ms)	Errors
I1	Baseline (default 8t)	74.4	171	286	330	0%
I2	+Indexes	74.6	184	276	314	0%
I3	Threads=16	83.0	34	238	287	0%
I4	Threads=24	77.6	59	582	881	0%
I5	Threads=32	91.6	37	75	111	0%

OpenLDAP Performance Test Report

Symas OpenLDAP 2.6.13 on RHEL 9 -- Master/Replica -- Confidential

I6	32t+LMDB+OS (100 conc)	93.6	632	758	981	0%
I7	Mixed load (login+write)	92.6	798	940	1014	~2%

BEST: 32 threads (I5) -- 91.6 ops/sec, p50=37ms, p95=75ms, p99=111ms.

This represents a 23% throughput increase and 78% latency reduction vs baseline.

NOTES:

- I6/I7 used 100 concurrent threads causing TLS handshake contention on the load-generator side. The LDAP server itself was not saturated.
- Optimal concurrency for this hardware is ~50 threads.
- True production throughput with persistent connections (HAProxy pool, ldap3 library) would be significantly higher than subprocess-based tests.

4. Write Load Test (500K Users)

A 500,000-user LDIF file (139 MB, 5.5M lines) was generated on the master in 0.9 seconds using Python, then stream-loaded via a single ldapadd -f session with TLS, measuring true online write performance.

Metric	Value
Users added	500,000
LDIF size	139 MB (5,500,000 lines)
Write method	ldapadd -f (single TLS session, streaming)
Write speed	390 adds/sec (average)
Peak write speed	420 adds/sec
Total load time	~21.4 minutes
Master DB before	100,431 entries (baseline)
Master DB after	600,001 entries
Master DB size after	637 MB (~1.06 KB/user)
LMDB usage	2.5% of 25 GB maxsize

Replication Sync During Write Load

Metric	Value
Replica sync rate	275 entries/sec (sustained)
Sync ratio	70.5% of master write rate
Sync lag at peak	~68,000 entries behind
Lag recovery time	~4 min after writes stop
Final state	93.6% synced within 5 min of load end
contextCSN match	Verified on both nodes

5. Stress Test Results

Concurrency	Ops/sec	p50	p95	p99	Notes
10 threads	~50	25ms	50ms	80ms	Underloaded
50 threads	84.8	33ms	240ms	342ms	Sweet spot
100 threads	93.6	632ms	758ms	981ms	TLS contention on LG
200 threads	92.6	798ms	940ms	1014ms	LG saturated

The LDAP server itself remained responsive at all concurrency levels. Latency degradation above 50 threads was caused by TLS handshake overhead on the load-generator side (subprocess-based tester creates a new TLS connection per operation). Production

OpenLDAP Performance Test Report

Symas OpenLDAP 2.6.13 on RHEL 9 -- Master/Replica -- Confidential

deployments should use persistent LDAP connections via HAProxy connection pooling or LDAP client libraries (python-ldap, ldap3) to avoid this bottleneck.

6. Optimized Configuration

6.1 LDAP Server Settings (cn=config via ldapmodify)

olcThreads	32
olcDbMaxSize (main)	26,843,545,600 (25 GB)
olcDbMaxSize (accesslog)	26,843,545,600 (25 GB)
olcDbConfig {0}	set_cachesize 0 536870912 1 (512 MB cache)
olcDbConfig {1}	set_flags writemap
olcDbConfig {2}	set_flags nometasync
olcDbConfig {3}	set_lg_regionmax 262144 (256 KB)
olcDbConfig {4}	set_lg_bsize 33554432 (32 MB)
olcDbCheckpoint	1024 30 (1 MB or 30 min)
olcConnMaxPending	256
olcConnMaxPendingAuth	1000
olcIdleTimeout	3600
olcLogLevel	none
olcAccessLogOps	writes (reduced from writes+reads+session)
olcAccessLogPurge	7+00:00 01:00:00 (7 days, every 1 hour)

6.2 Indices

uid eq,sub | mail eq | cn eq | entryUUID eq | entryCSN eq | member eq | objectClass eq

Applied via ldapmodify, then slapindex rebuilt offline (service restart required).

6.3 Kernel Tuning (/etc/sysctl.d/99-openldap-perf.conf)

net.core.somaxconn	65535
net.ipv4.tcp_max_syn_backlog	65535
net.ipv4.ip_local_port_range	1024 65535
net.ipv4.tcp_tw_reuse	1
net.ipv4.tcp_fin_timeout	15
net.core.netdev_max_backlog	65535
fs.file-max	2097152
fs.nr_open	2097152
vm.swappiness	1
vm.dirty_ratio	5
vm.dirty_background_ratio	2

6.4 Systemd Limits (limits.conf drop-in)

LimitNOFILE=1048576
LimitNPROC=65536
LimitMEMLOCK=infinity
TasksMax=65536

7. Deployment Procedures

7.1 Master Tuning (bank-tune-master.sh)

```
sudo bash bank-tune-master.sh          # Apply all tuning
sudo bash bank-tune-master.sh --dry-run # Preview changes
sudo bash bank-tune-master.sh --verify  # Check current settings
sudo bash bank-tune-master.sh --rollback # Restore from backup
```

The script performs 11 steps: backup, LMDB sizing, indices + slapindex, LMDB performance flags, checkpoint, threading, accesslog fix, log level, kernel parameters, systemd limits, restart + verification.

7.2 Replica Tuning (bank-tune-replica.sh)

```
sudo bash bank-tune-replica.sh          # Apply all tuning
sudo bash bank-tune-replica.sh --dry-run # Preview changes
sudo bash bank-tune-replica.sh --verify  # Check current settings
sudo bash bank-tune-replica.sh --rollback # Restore from backup
```

Additional replica-specific steps: ppolicy module check, read ACL, syncrepl verification, contextCSN check.

7.3 Important Notes for Bank Deployment

- Source PATH before running: `source /opt/symas/etc/openldap/sysmas_env.sh`
(Bank path is NOT `/etc/profile.d/symas_env.sh`)
- `ldapsearch` always use `-o ldif-wrap=no` to avoid line wrapping
- Use "replace" not "add" for existing `olc` attributes
- `olcDbMaxSize` takes effect immediately -- no restart required
- Verify `slapd -Tt` passes before/after changes
- If `accesslog` purge is broken, may need manual offline truncation
- Never use `slappasswd --use Python SSHA (hashlib.sha1 + os.urandom + base64)`
- Log level must be "none" in production at 500K+ users

8. Test Scripts and Framework

File	Purpose
script/perf/bank-tune-master.sh	Production master tuning (apply/verify/rollback)
script/perf/bank-tune-replica.sh	Production replica tuning (apply/verify/rollback)
script/perf/tune-comprehensive.sh	Full lab tuning (all settings in one pass)
script/perf/tune-ldap.sh	Iterative tuning (step-by-step)
script/perf/ldap-load-tester.py	Python concurrent LDAP load generator
script/perf/monitor-ldap.sh	Real-time <code>cn=Monitor</code> + OS metrics collector
script/perf/run-perf-suite.sh	Full test suite orchestrator
script/perf/gen-ldif.py	LDIF generator (1.5M+ users)
script/perf/bulk-load.sh	Offline <code>slapadd</code> + replica sync
script/perf/deploy-perf.sh	Master+replica TLS deployment
script/perf/ips.env	Lab IP addresses and credentials
terraform/openldap-perf-test/	Terraform: VPC, EC2, security groups, JMeter

9. Infrastructure (Terraform)

Resource	Spec	IP / ID
----------	------	---------

OpenLDAP Performance Test Report

Symas OpenLDAP 2.6.13 on RHEL 9 -- Master/Replica -- Confidential

VPC	10.40.0.0/16	vpc-04430038ae109ba55
Subnet A	10.40.1.0/24 (us-west-2a)	subnet-023d2a7eb2b574bd9
Subnet B	10.40.2.0/24 (us-west-2b)	subnet-07af0894df5d785e0
Perf Master	m5a.4xlarge, 50GB gp3	34.208.189.170 (i-0b41c7922e1ebd44b)
Perf Replica	m5a.4xlarge, 50GB gp3	35.88.141.50 (i-0511da8c7701e264b)
Load-gen 1	t3.medium, JMeter 5.6.3	44.247.77.105 (i-0992993a0cd45066f)
Load-gen 2	t3.medium, JMeter 5.6.3	35.94.20.201 (i-0a776e3a90cf5a24a)
Terraform module	terraform/openldap-perf-test/	Git: feature/perf-test-tuning

10. Capacity Planning and Projections

Based on measured performance with 600K users (637 MB DB):

Scenario	Data Size	Load Time	Read Capacity
500K users	~530 MB	21 min (online)	~92 ops/sec (50 conc)
1.0M users	~1.1 GB	43 min (estimated)	~92 ops/sec
1.5M users	~1.6 GB	64 min (estimated)	~92 ops/sec
2.0M users	~2.1 GB	85 min (estimated)	~92 ops/sec

NOTES:

- LMDB 25 GB max size supports ~23M entries at ~1 KB/user
- Read throughput scales with threads up to 32 (diminishing returns beyond)
- Write throughput is I/O bound; writemap+nometasync flags critical
- Replica sync at 275/sec can handle master writing at up to ~390/sec
- For >1000 concurrent connections, deploy HAProxy TCP load balancer (reads -> replica, writes -> master)

11. Key Recommendations

1. DEPLOY tuning scripts on both master and replica (use --dry-run first)
2. SET log level to "none" in production -- "stats sync" at 500K users is expensive
3. ADD all 7 indices -- unindexed searches cause full table scans
4. ENSURE writemap+nometasync LMDB flags -- dramatically reduce I/O
5. MONITOR replica contextCSN after bulk imports to confirm sync
6. USE ldapadd -f for bulk loads -- single TLS session, 390/sec vs 0.69/sec per-entry
7. DEPLOY HAProxy for production read scaling: writes -> master, reads -> replica
8. SET 25 GB LMDB max size on both nodes -- prevents MDB_MAP_FULL
9. SOURCE /opt/symas/etc/openldap/symas_env.sh (bank path, not /etc/profile.d/)
10. VALIDATE config with slapd -Tt before every restart

Report generated from live lab tests on AWS us-west-2, 2026-06-14.

Contact: Ibrahim Timor <ibrahim.timor@nxtedgetechnologies.com>